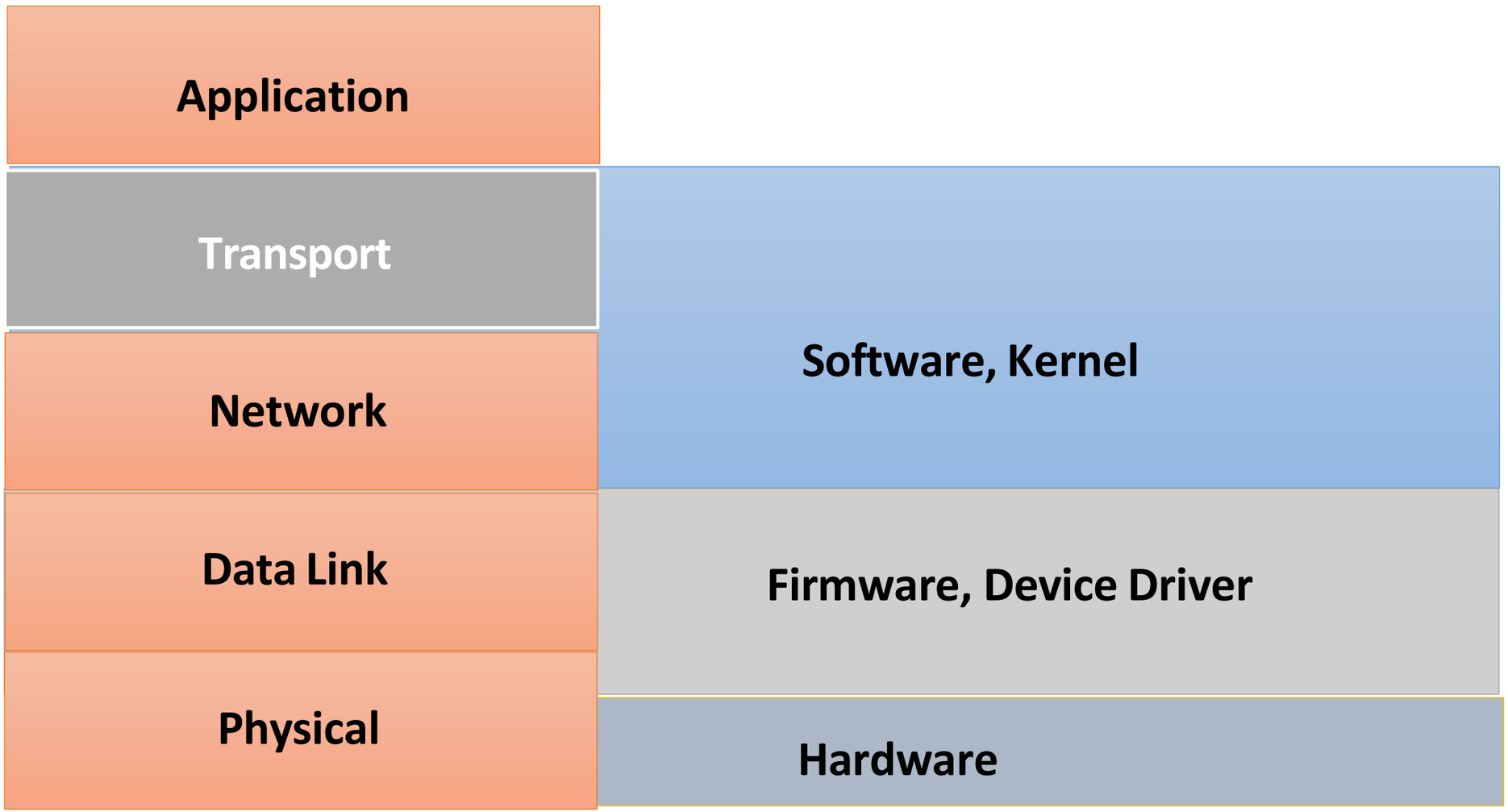
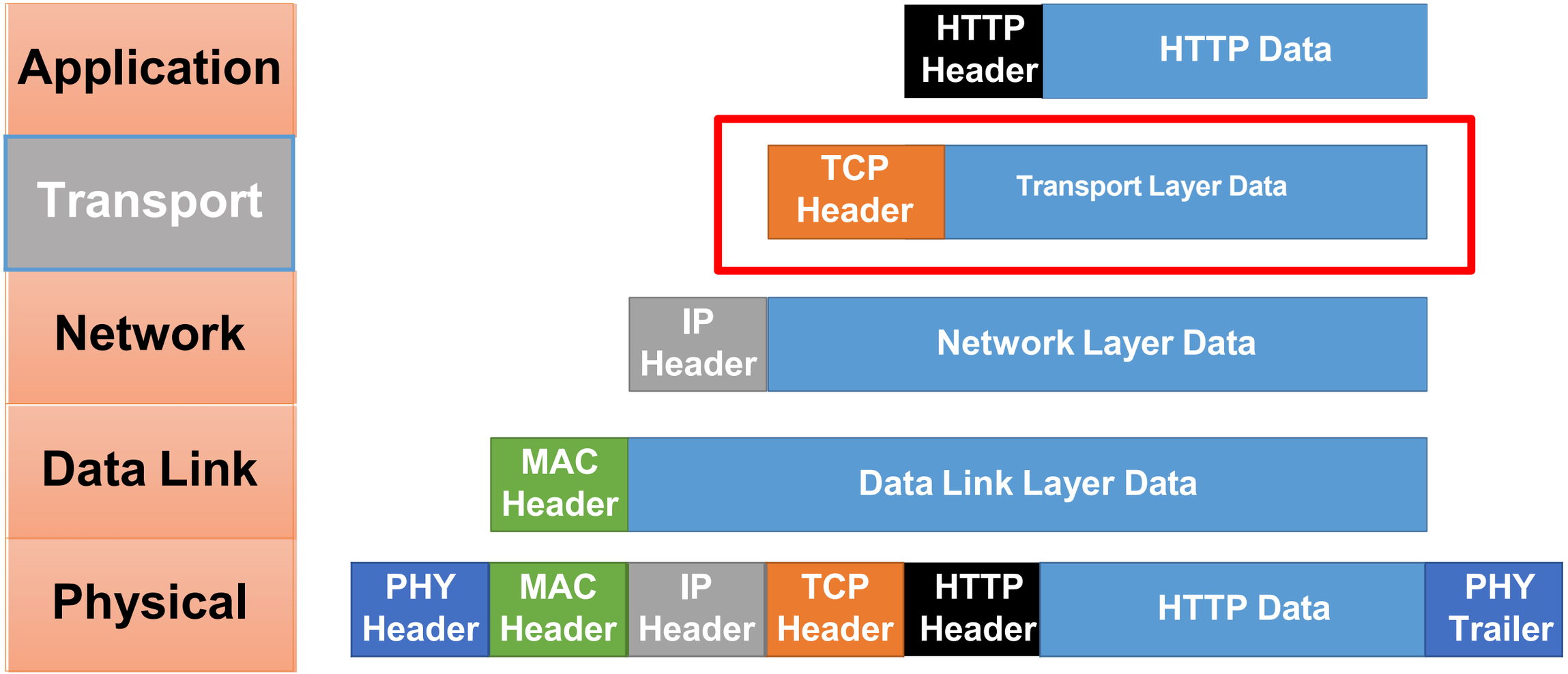
Transport Layer - I

[The Transport Layer Services and Principles of Connection Management]

Protocol Stack Implementation in a Host



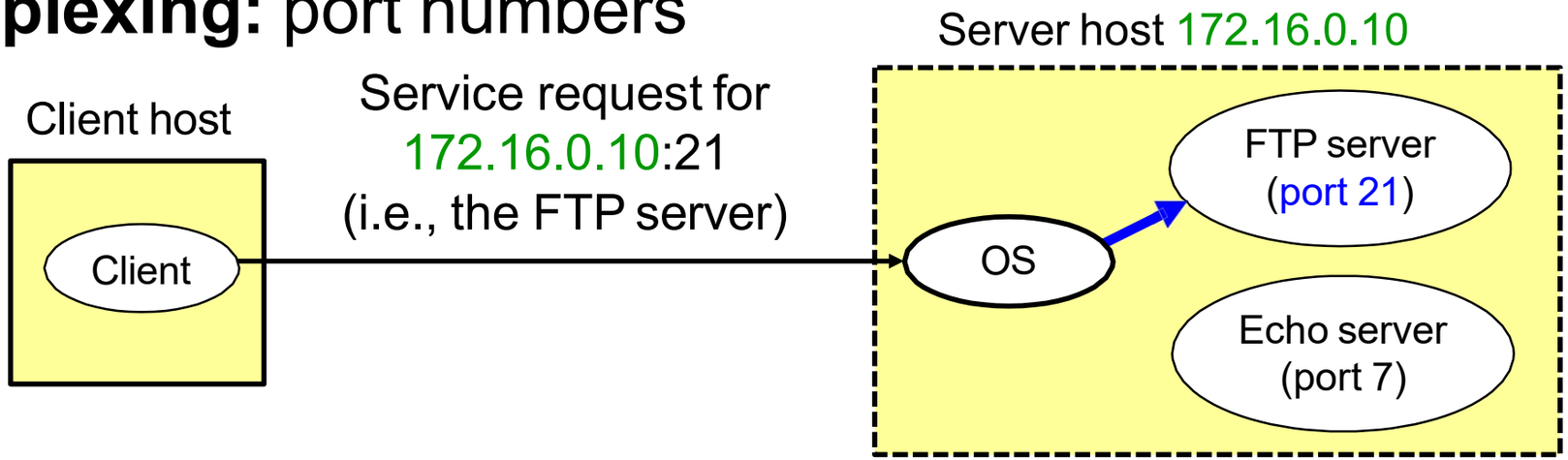
The Layered Abstraction



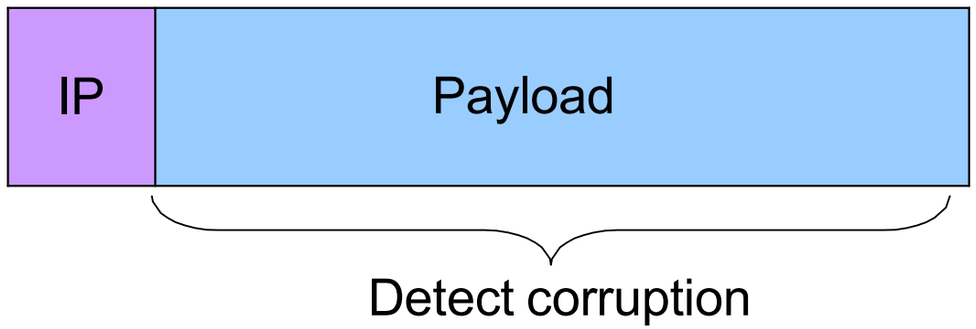
Illustrative example: HTTP over TCP/IP

Two Basic Transport Layer Services

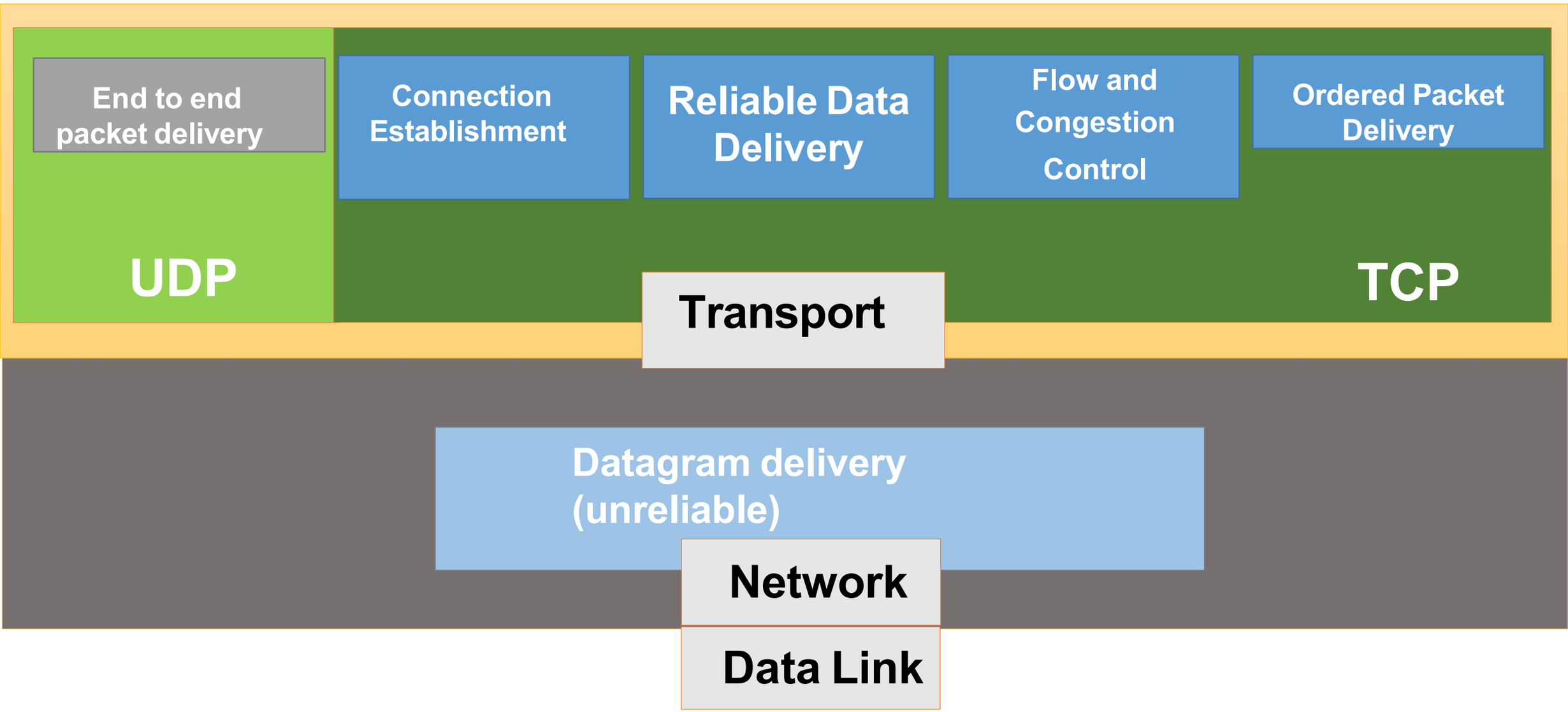
- **Demultiplexing:** port numbers



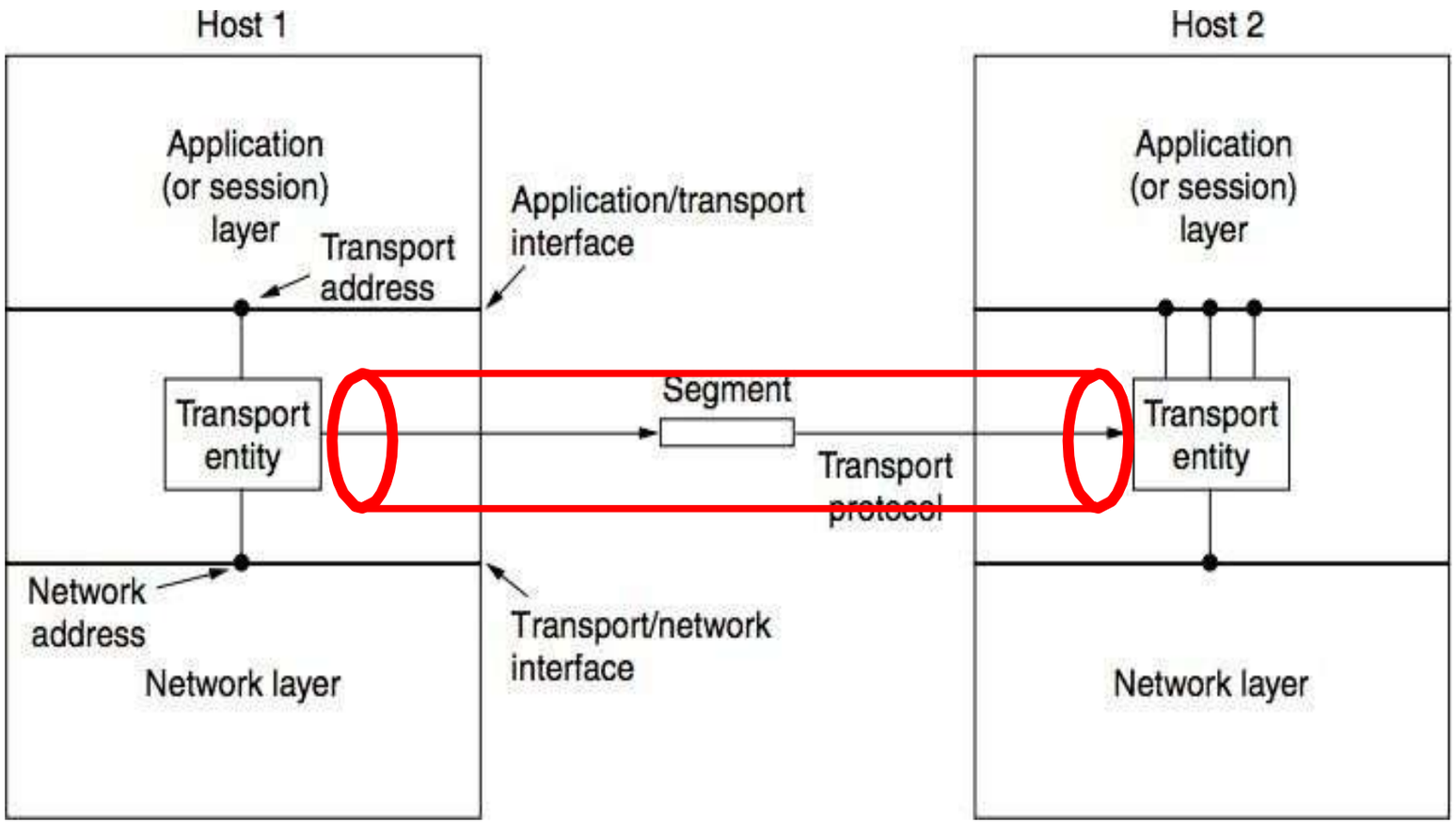
- **Error Detection:** Checksum



Transport Layer Services



Transport Layer - Interfacing with Application and Network

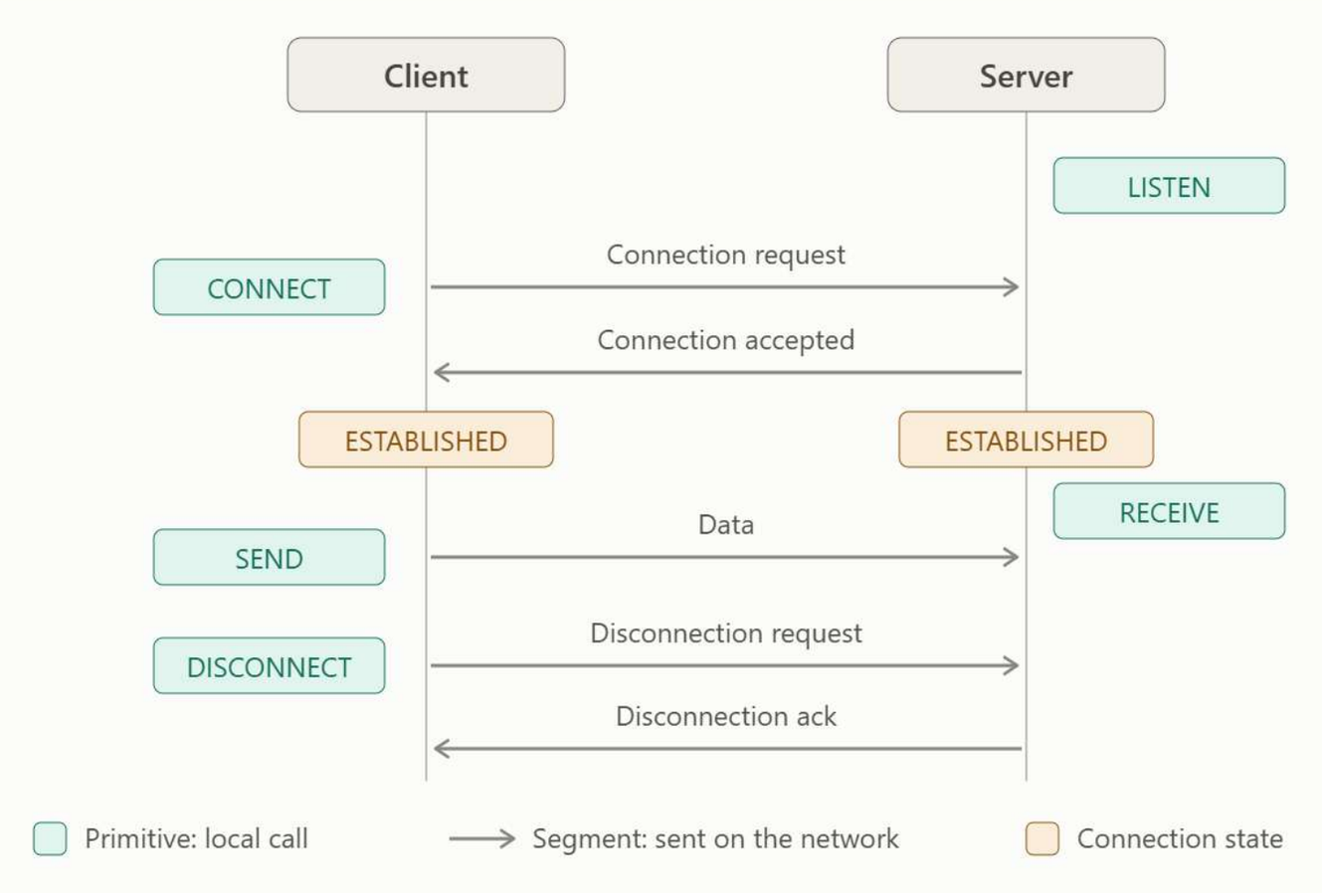


Create a logical pipe between the sender and the receiver and monitor the data transmission through this pipe

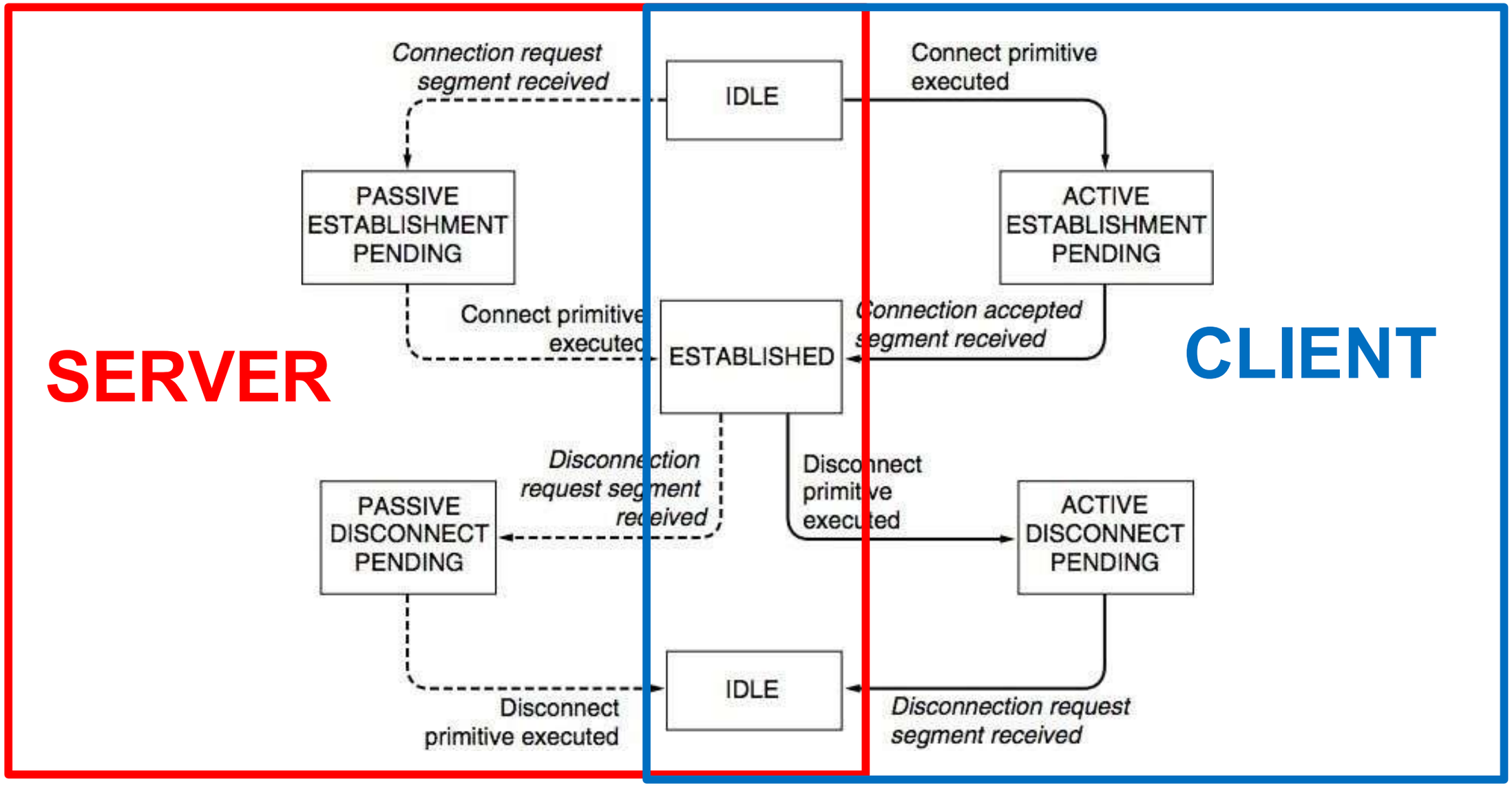
Transport Service Primitives

- A connection-oriented transport protocol must remember the state of each connection, so that the right action can be taken when a segment arrives. The service it offers the application is a small set of primitives.

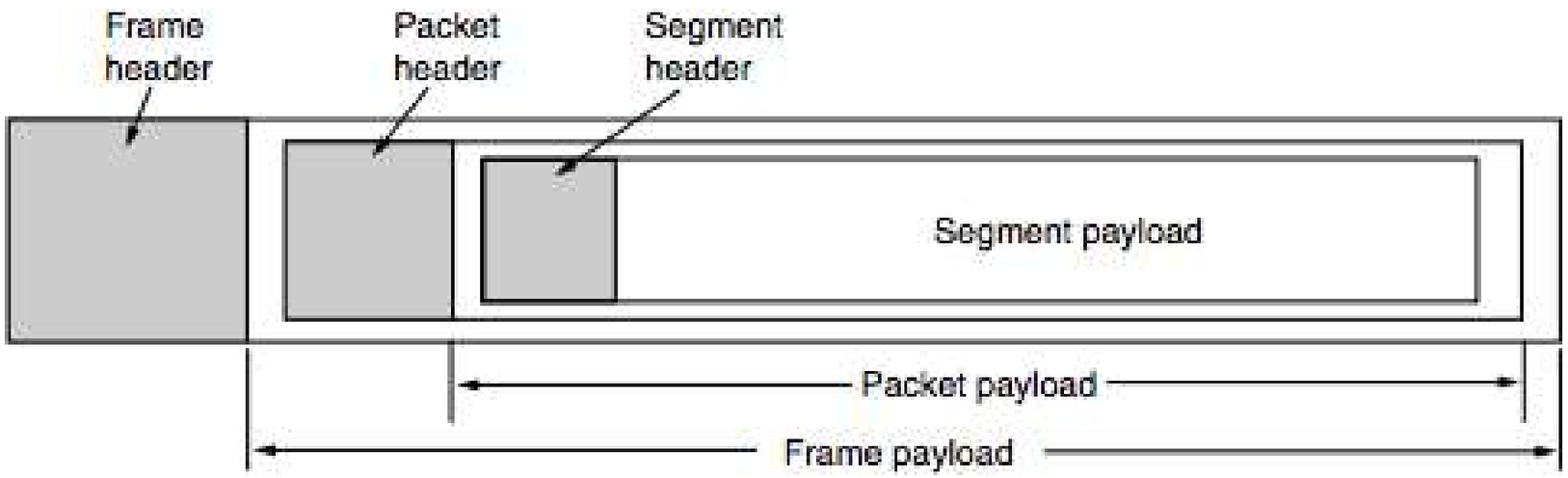
This simple two-message setup is enough when the network is well behaved. We will see shortly that it breaks when the network delays and duplicates packets, which is why real protocols use a three-way handshake.



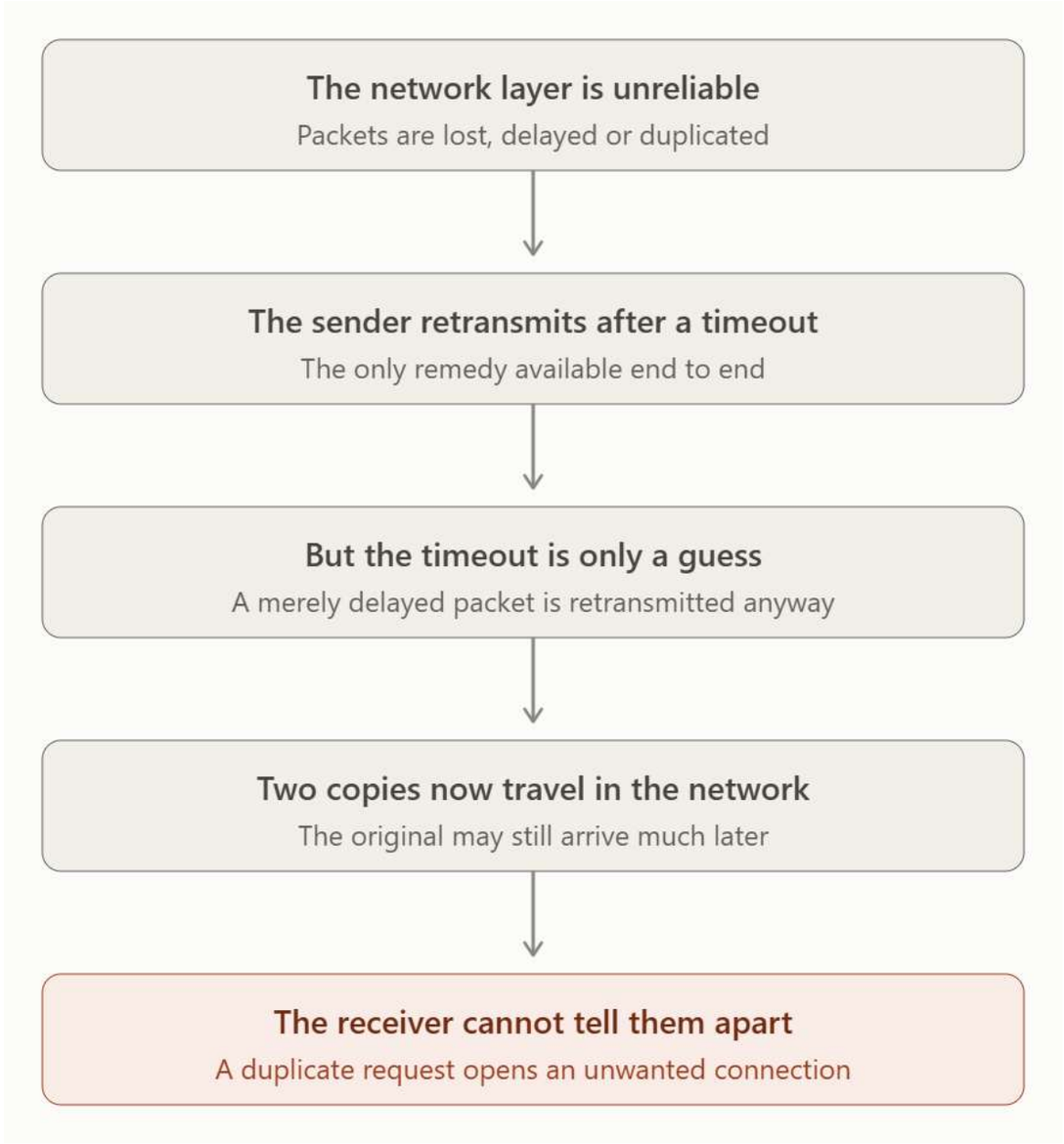
Transport layer protocols: statediagram



Segment, Packet (or Datagram) and Frame

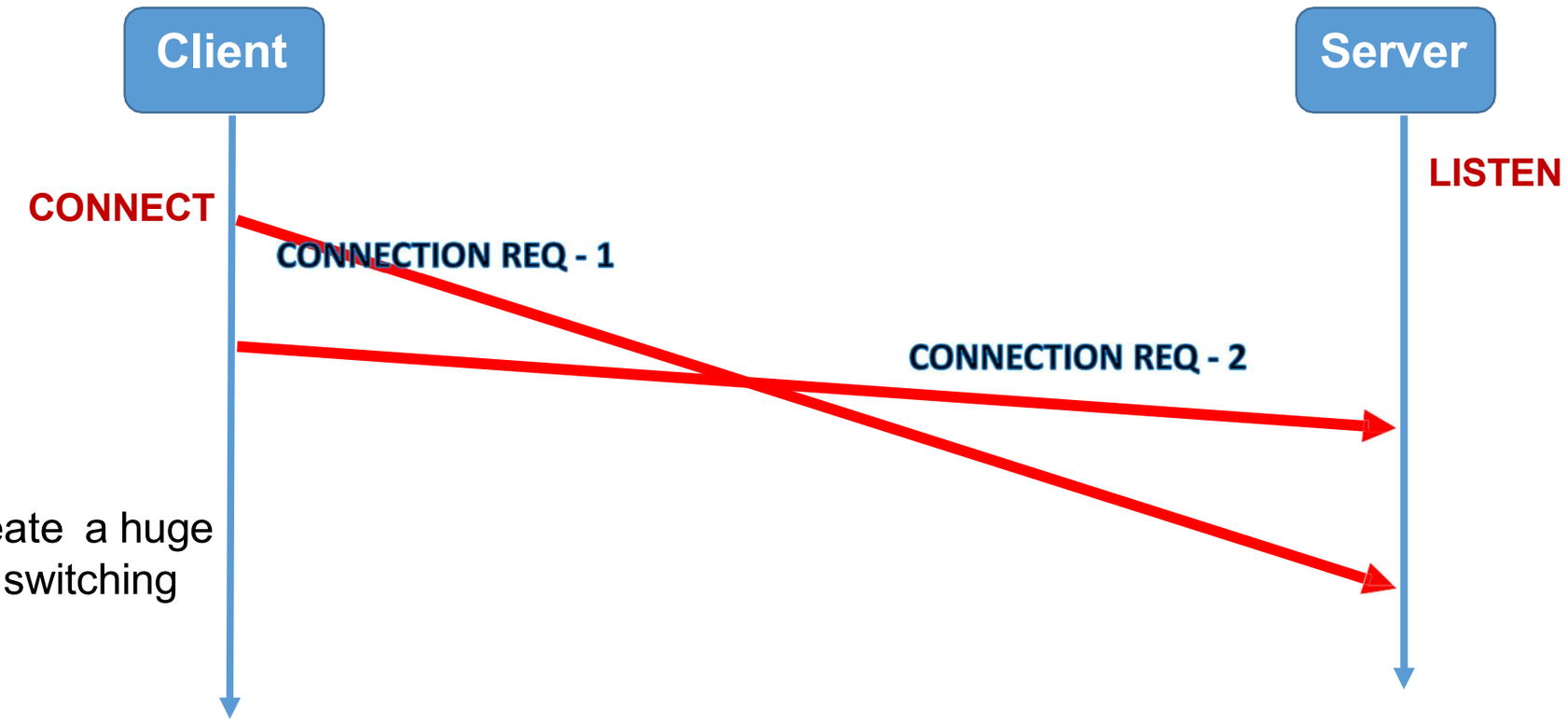


Why an unreliable network makes connection setup hard?



- For ordinary data a duplicate is harmless, because the connection's sequence numbers let the receiver discard it.
- For a connection request there is no such context yet, because the request is the message that establishes it.

When the basic primitives for connection establishment fails?

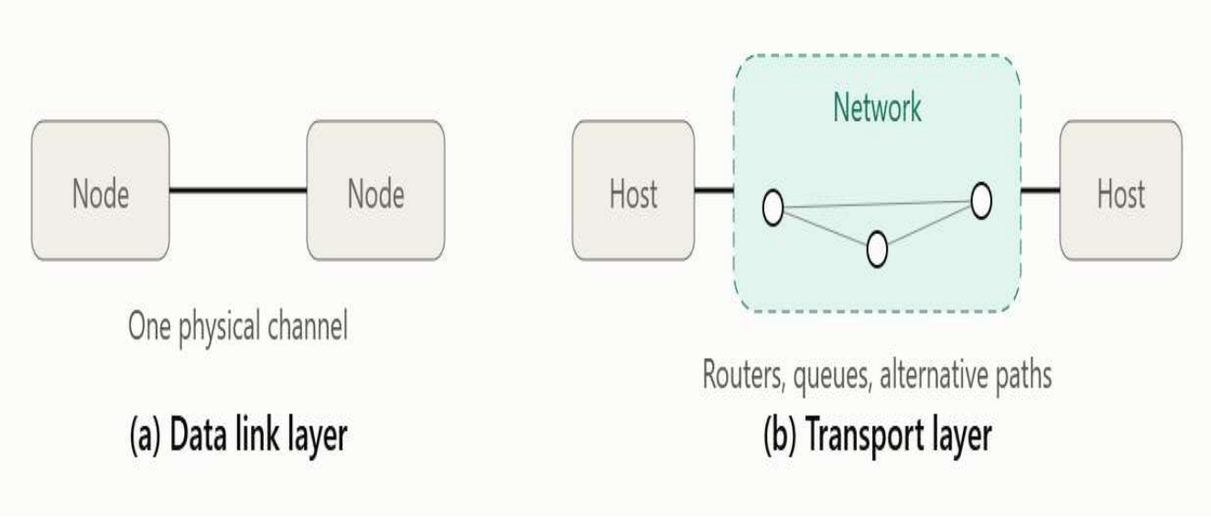


Delayed duplicates create a huge confusion in the packet switching network.

REQ-2 arrived first and a connection was set up. REQ-1 now arrives. Is it new, or a stale copy?

Why transport is harder than the link layer? Differences with Link Layer

- At the data link layer, two directly connected nodes talk over a physical channel. At the transport layer, that channel is replaced by the entire network. Three consequences follow.



1. **The peer must be addressed explicitly.** On a link the peer is already present: implicit on a point-to-point link, or named by a MAC address on a shared medium. At the transport layer the peer is somewhere in the network, so every segment must carry an explicit transport address. A connection-oriented protocol must go further and establish the connection before data can flow.
2. **The network can store packets.** A link delivers a packet after a short, bounded delay or loses it. It cannot hold a packet and release it after later ones. A network can, through queueing and rerouting, so an old packet may surface long after the sender gave up on it.
3. **Delay varies, and so does the number of connections.** A link has one peer and a roughly fixed round-trip time. A transport entity holds many connections at once, over paths whose delay and available bandwidth fluctuate as connections compete. Buffering and timeout values cannot be fixed constants.

Handling delayedduplicates

- We cannot stop packets from being duplicated and delayed. But when it happens, they must be rejected as duplicates and not processed as fresh packets. Two ways to attack this, neither satisfactory.

Solution 1: throwaway transport addresses

- Each time a transport address is needed, a brand new one is generated. When a connection is released, that address is discarded and never used again.
- Never reusing transport addresses, at either end, prevents old packets from affecting new connections. However, it leaves unresolved how clients discover the fresh address.

Solution 2: a unique connection identifier

- The initiating party gives each connection a sequence number, incremented for every connection it establishes, and puts it in every segment, including the connection request.
- After a connection is released, each transport entity records it in a table of obsolete connections as a (peer transport entity, connection identifier) pair. An arriving connection request is checked against that table.
- Why it fails: each transport entity must keep this history effectively indefinitely, at both the source and the destination. If a machine crashes and loses its memory, it no longer knows which connection identifiers its peers have already used.

Handling delayed duplicates: restrict packet lifetime

- **Solution 3:** Rather than allowing packets to live forever within the network, devise a mechanism to kill off aged packets that are still hobbling about. This makes a feasible solution possible.

- **Three ways to restrict packet lifetime to a known maximum**
- **Restricted network design.** Prevent packets from looping, and bound the maximum delay on every path, including time spent queued during congestion. (Difficult, because it constrains how the network may be built and operated.)
- **A hop count in each packet.** Initialize it to a maximum value and decrement it at every hop. When it reaches zero, the packet is discarded. This is the most feasible of the three and is what IP actually does, in the TTL field.
- **A timestamp in each packet.** Each packet carries the time it was created, and a router discards it once a defined lifetime has elapsed. This requires the clocks on every router to be synchronized, which is a strong requirement.

- **Design Challenge:** We need to guarantee not only that a packet is dead, but also that all acknowledgements to it are also dead. That is, define a maximum packet lifetime, T.

A maximum packet lifetime, T

- Let us define a maximum packet lifetime T: if we wait a time T secs after a packet has been sent, we can be sure that all traces of it (packet and its acknowledgement) are now gone.
- The heart of the method is for the source to label segments with sequence numbers that will not be reused within T secs.
    - T and the packet rate together determine how wide the sequence number field must be. A faster sender consumes numbers faster, so it needs more bits.
    - Only one packet carrying a given sequence number can be outstanding at any time. A late arrival carrying that number is therefore recognizable as stale.

The crash case

A machine that crashes loses all memory of where it was, including which sequence numbers it had used. After recovery it cannot tell an old segment from a new one.

One possibility: require a transport entity to stay idle for T seconds after recovery. The idle period lets all old segments die off, after which the sender can start again with any sequence number.

Why that is unattractive

In a complex internetwork T may be large, so a host would be unusable for a long time after every restart.

We need a way to recover immediately, without relying on memory that the crash destroyed.

Tomlinson's Clock: Sequence Numbers from Time

Instead of idling for T seconds after a restart, derive sequence numbers from a time-of-day clock.

Four required properties

1. Clocks at different hosts need not be synchronized

1. Each side picks its own initial sequence number and validates it through the ACK it receives

2. Each clock is a binary counter incrementing at uniform intervals

1. Gives a linear relation between time and initial sequence number

3.Counter width is at least the sequence number width

1. The low-order k bits of the clock become the k-bit initial sequence number

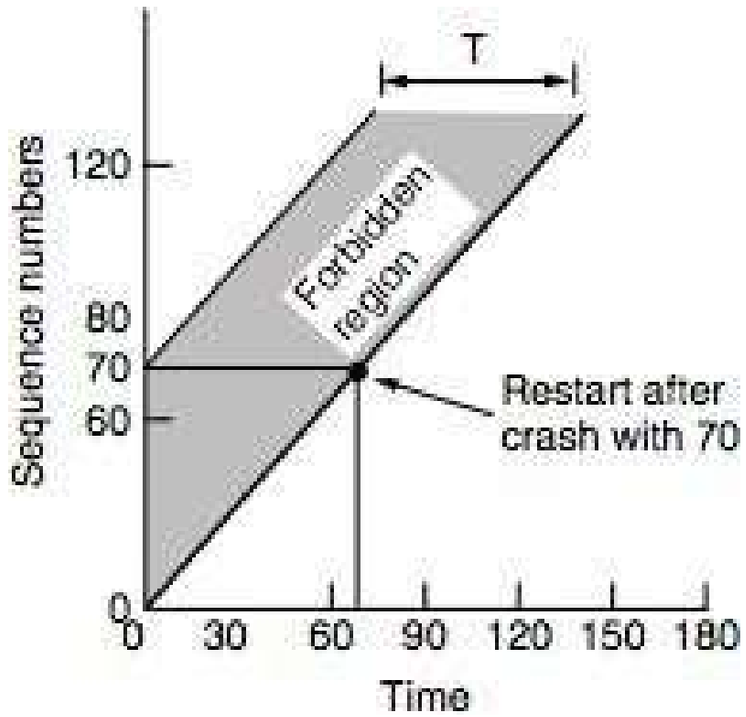
4.The clock keeps running even if the host goes down

1. After a crash the clock has moved past every number the host used, so no memory is needed

No table, no idle period, no synchronization. The clock supplies what memory cannot.

Choosing the initial Sequence Number

- When a connection is set up, the low-order *k* bits of the clock are used as the *k*-bit initial sequence number
- The sequence space should be so large that by the time sequence numbers wrap around, old segments with the same sequence number are long gone.
- Once both transport entities have agreed on the initial sequence number, any sliding window protocol can be used for data flow control.
- This window protocol will correctly find and discard duplicates of packets after they have already been accepted.



Linear relation between time and initial sequence numbers

How to ensure that packet sequence numbers are out of the Forbidden Region?

A connection's sequence numbers can enter the forbidden region from either direction.

1.Sending too fast on a newly opened connection

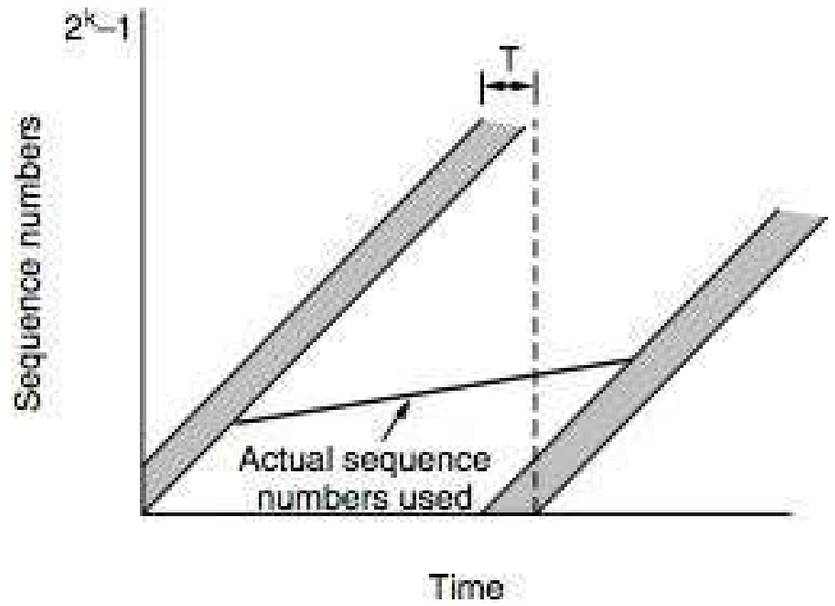
1. The numbers in use climb faster than the clock advances, so the connection catches up with the region from below

2.Sending too slowly

1. The connection stays open so long that the clock advances and the region catches up with the numbers still in use

3.The rule that prevents the first case

1. Limit the data rate to one segment per clock tick
2. The sequence numbers in use can then never outrun the clock that issues them
3. No two packets with the same sequence number are ever in the network at once



What the Clock Does Not Solve:

The clock lets a host choose a fresh initial sequence number without remembering anything. The receiver still has no way to judge whether an arriving request is current.

1.What the clock solved

1. A host can pick an initial sequence number that no live segment is using, even immediately after a crash

2.What remains

1. The receiver keeps no record of sequence numbers, so a connection request arriving on its own cannot be checked against anything

3.The fix: a three-way handshake

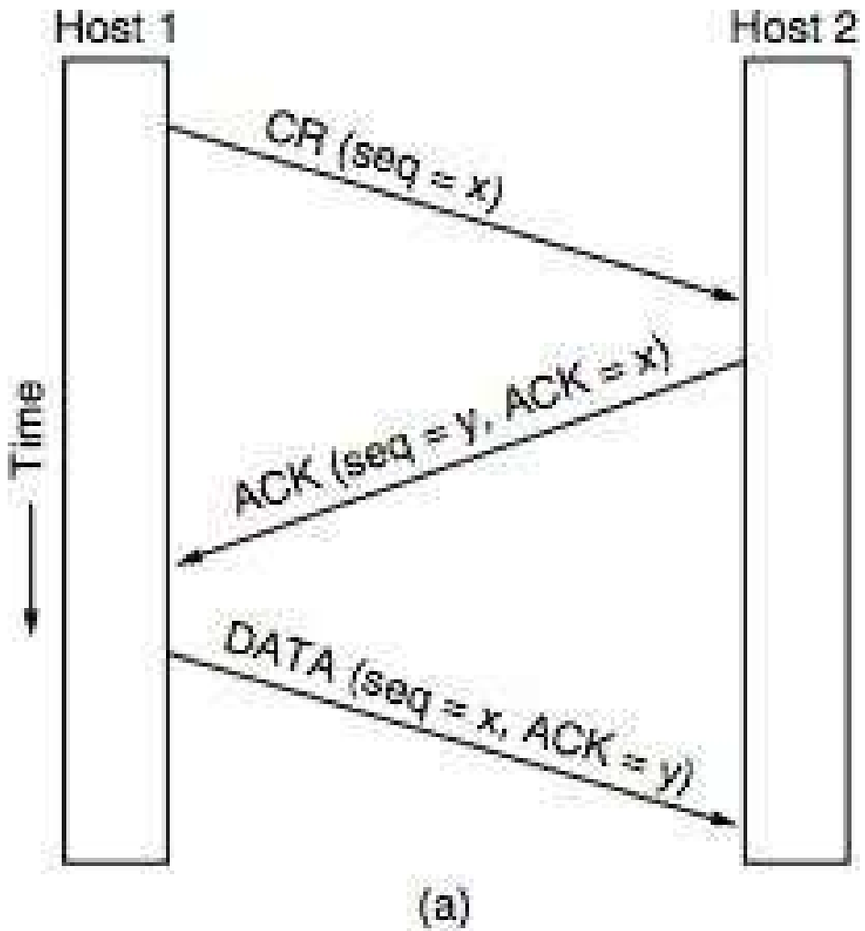
1. Each side's chosen number is echoed back by the peer
2. Each side validates its own number by looking at the acknowledgement

4.The result

1. Positive synchronization: neither side proceeds until it has seen its own number confirmed

This is where the agreement on initial sequence numbers, assumed earlier, is actually reached.

Three Way Handshake



1.Host 1 validates x

1. The reply echoes x. Host 1 sent x a moment ago, so a reply carrying it confirms the exchange is current and not the residue of an old connection

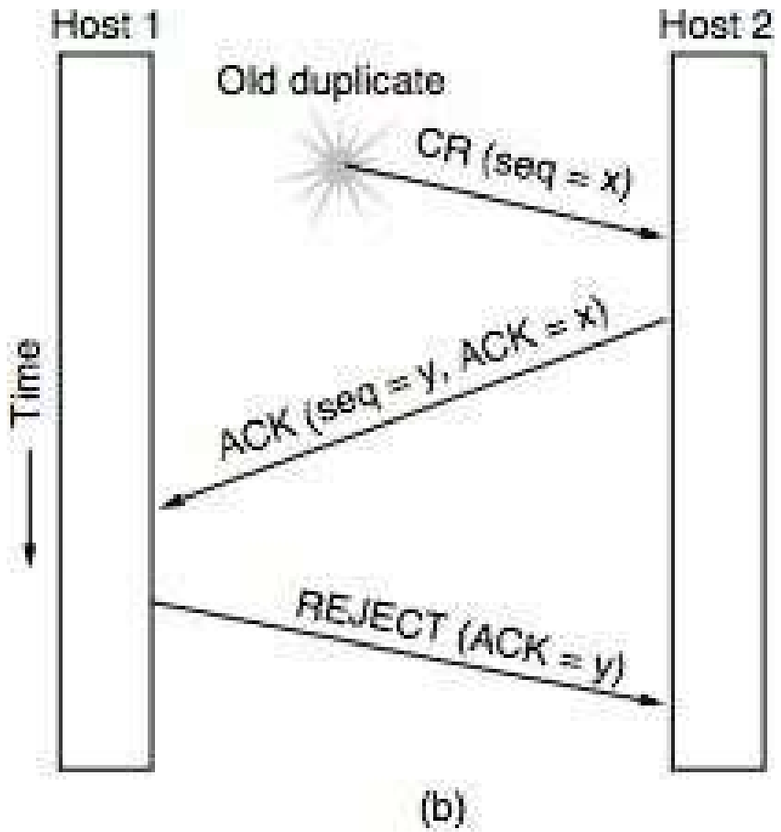
2.Host 2 validates y

1. The third message echoes y. Host 2 only commits to the connection once it sees its own number confirmed

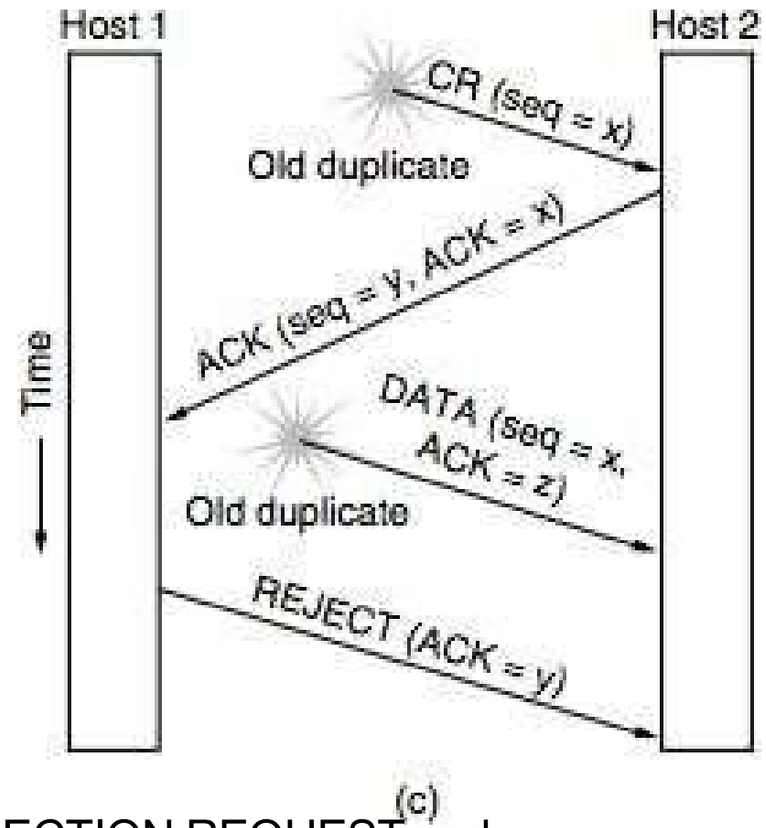
3.Why three messages and not two

1. Host 2 cannot judge a lone request. Only the third message tells it that a live peer chose this exchange
2. This is the positive synchronization.

Three Way Handshake – Handling Delayed Duplicate



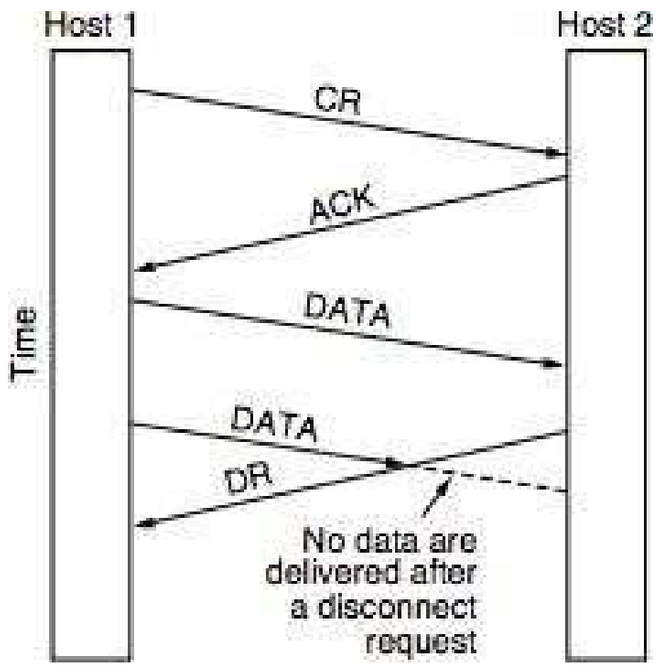
CONNECTION REQUEST is a Delayed Duplicate



CONNECTION REQUEST and ACKNOWLEDGEMENT both are Delayed Duplicates

Connection Release: Asymmetric Release

Asymmetric release follows the telephone model: when one party hangs up, the connection is gone in both directions.



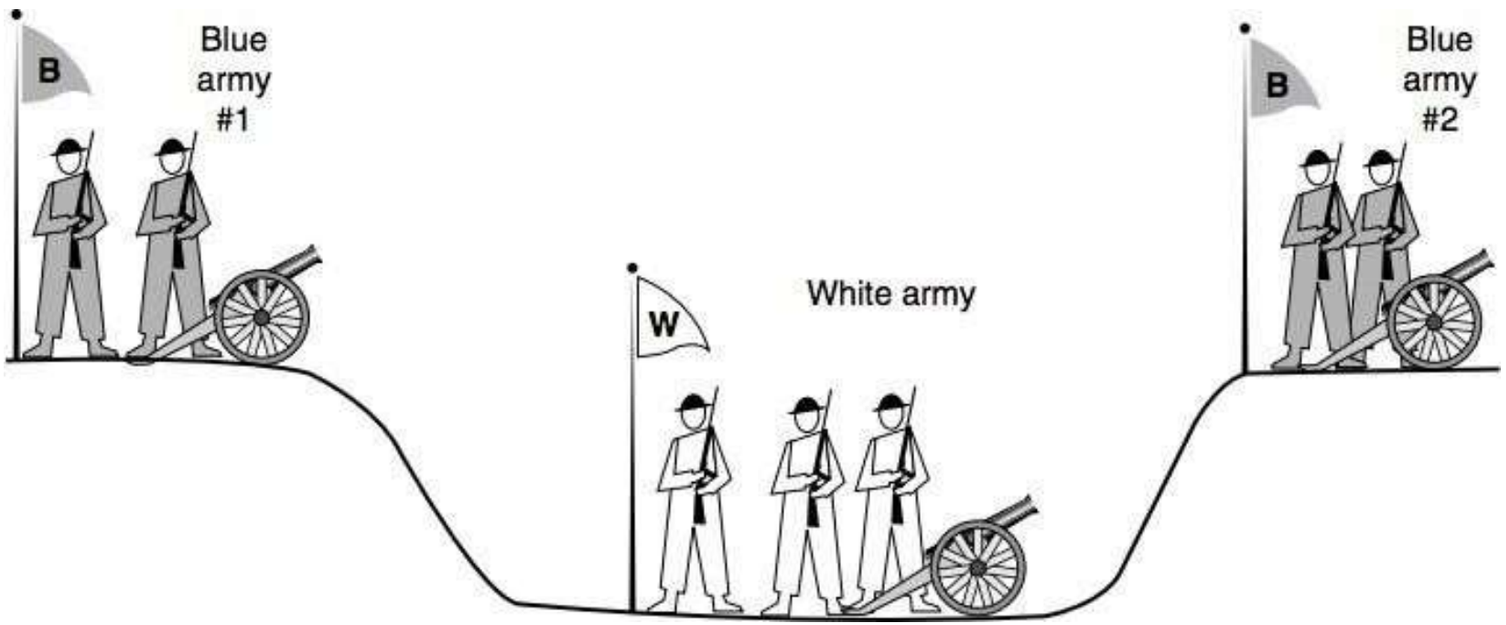
- When one side breaks connection abruptly, the connection is broken
- This may result in data loss
- Abrupt release is simple but unsafe. Treating the connection as two independent one-way connections, each released separately, is the alternative.

Connection Release: Symmetric Release

Treats the connection as two separate unidirectional connections and requires each one to be released separately

1. Each direction closes on its own
    1. A host that has finished sending releases its own direction
    2. It can still receive, because the peer's direction is untouched
2. When this is enough
    1. It works cleanly when each side can determine on its own that it has nothing more to send, and signals this to its peer.
    2. Each side simply announces that it is done
3. The obvious protocol
    1. Host 1: "I am done"
    2. Host 2: "I am done too"
4. The question
    1. Does this always work?

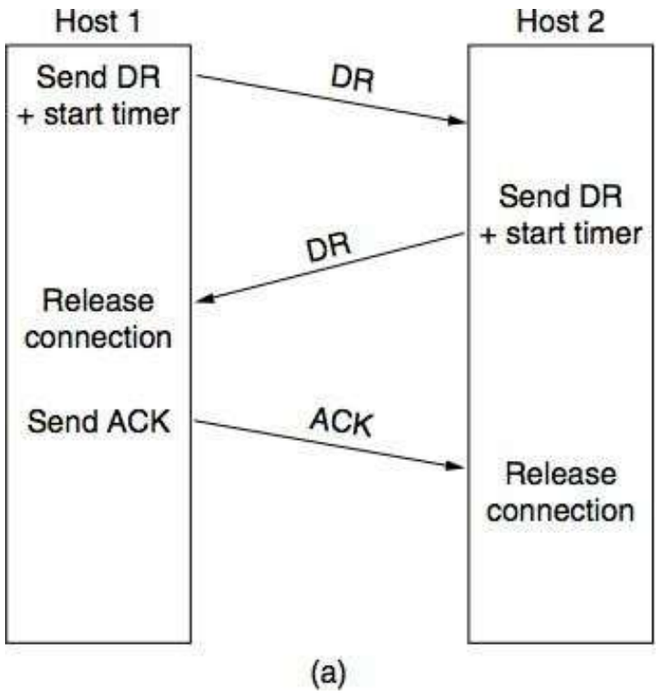
The Two Army Problem



No protocol exists to solve this

Let every party take individual decision

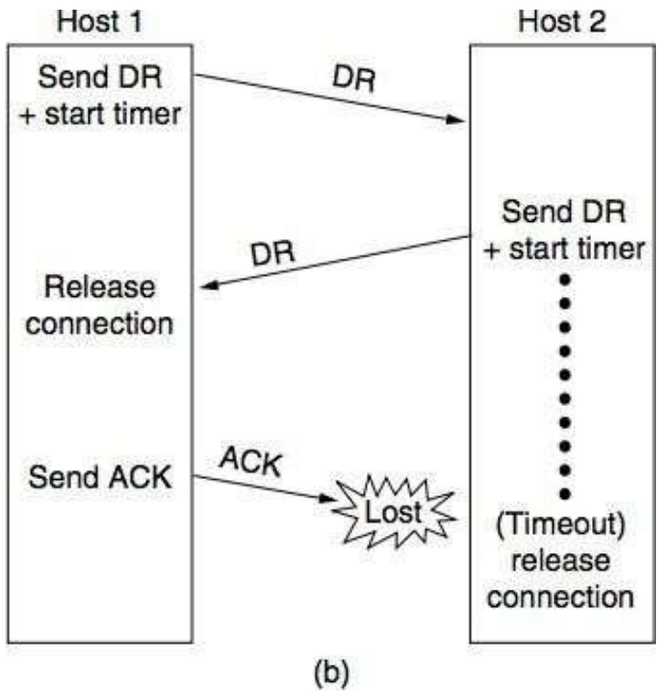
Connection Release



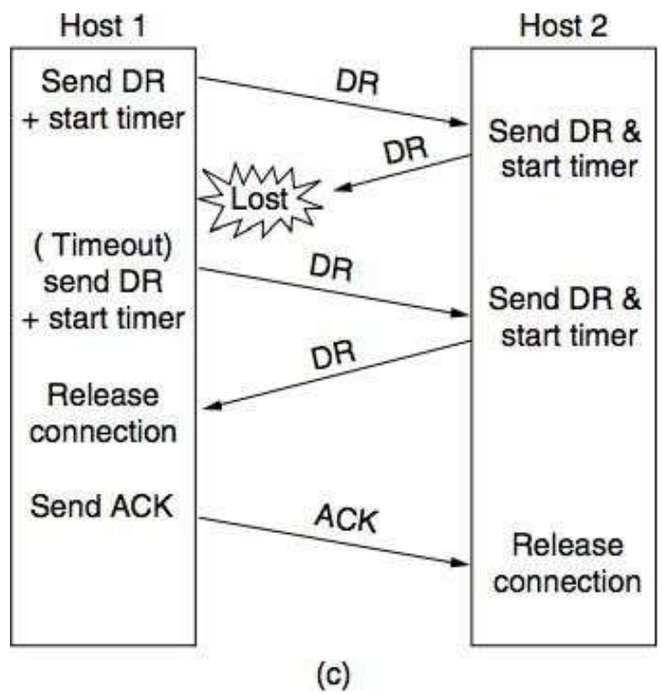
- Normal release sequence, initiated by transport user on Host 1

- DR=Disconnect Request
- Both DRs are ACKed by the other side

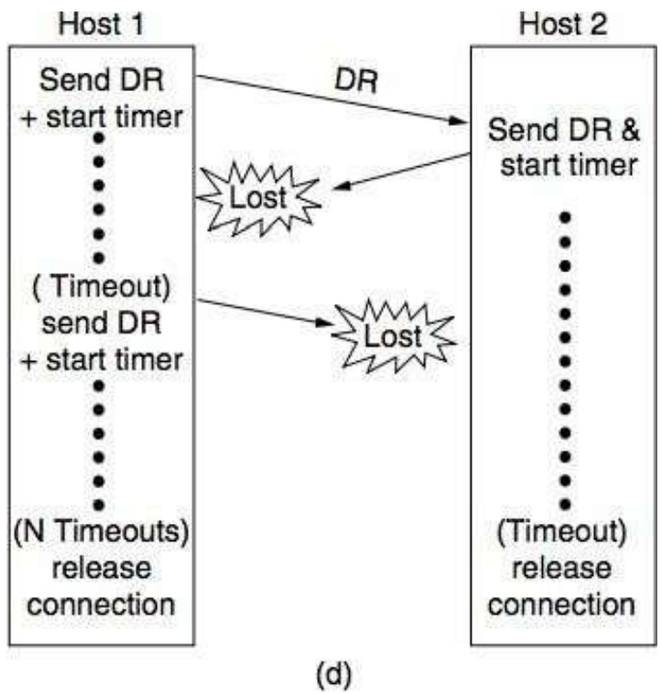
Connection Release: When Messages are Lost



Final ACK lost, Host 2 times out



Lost DR causes retransmissions



Extreme: Many lost DRs cause both hosts to timeout

Next we will look into TCP & UDP